



实验报告

课程 嵌入式系统及应用

题目 GPIO 输出摩尔斯电码灯光实验

院系名称

专业 _____ 班级 _____

学生姓名 _____ 学号 _____

任课教师

时 间

一、实验设备与软件环境

设备:

- ① PC 电脑, 至少需要 2 个 Type-A 的 USB 插口;
- ② ALIENTEK MiniSTM32 实验套件;
- ③ miniIO-V2 接口板;
- ④ 杜邦线

环境:

macOS 26.3; CLion 2025.3; CMake、OpenOCD、arm-none-eabi-gdb 工具链

二、实验任务

基于 LED 发光的摩尔斯电码输出:

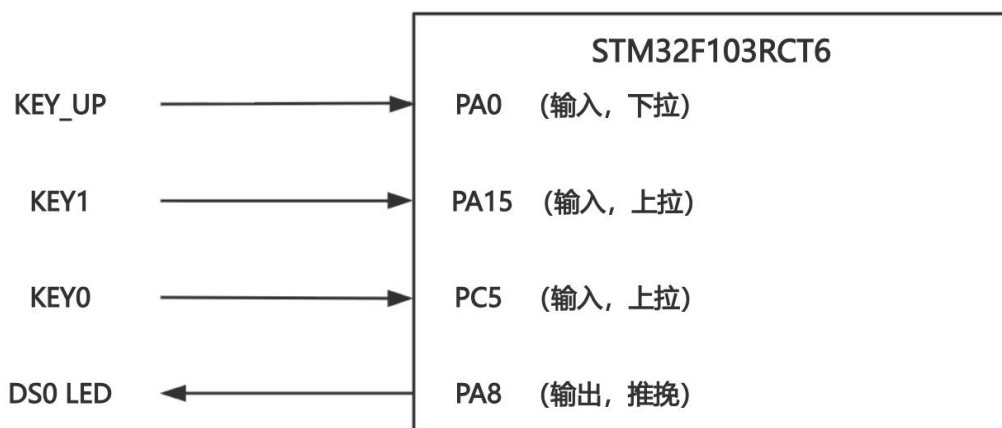
- ① 按下 KEY_UP 发送一次 “SOS”
- ② 按下 KEY1 发送一次 “YES”
- ③ 按下 KEY0 发送一次 “NO”

三、实验原理图及分析

3.1 实验涉及的 GPIO 与外设连接关系

结合开发板资料与实际工程代码, 本实验最终使用了 1 个 LED 输出端和 3 个按键输入端。MiniSTM32 开发板上, WK_UP 对应 PA0, KEY1 对应 PA15, KEY0 对应 PC5, 板载蓝色 LED DS0 对应 PA8。指导书同时给出 DS1 接在 PD2, 但本次实际工程代码只使用了 DS0 (PA8) 作为灯光输出口。另一个需要注意的点是, PA15 默认与 JTAG 复用, 因此若要把 PA15 当作普通按键输入, 必须先关闭 JTAG 占用, 只保留 SWD 调试功能。

3.2 实验连线示意图



3.3 各 GPIO 初始化模式分析

本实验中 4 个关键 GPIO 的初始化方式如下：

引脚	对应器件	实际工程配置	作用说明
PA8	DS0 LED	推挽输出	控制摩尔斯灯光明灭
PA0	WK_UP	下拉输入	按下后检测高电平，发送 “SOS”
PA15	KEY1	上拉输入	按下后检测低电平，发送 “YES”
PC5	KEY0	上拉输入	按下后检测低电平，发送 “NO”

STM32 的 GPIO 可配置为输入浮空、输入上拉、输入下拉、模拟输入、开漏输出、推挽输出、复用推挽、复用开漏等 8 种模式。

本实验只实际使用了其中 3 种：推挽输出、输入下拉、输入上拉。指导书还给出了 GPIO 读写常用方式：输入读取通常通过 `GPIO_ReadInputDataBit()` 完成，输出控制通常通过 `GPIO_SetBits()` 与 `GPIO_ResetBits()` 完成。

3.4 LED 发光控制

指导书中的 LED 原理图表明，DS0/DS1 与 3.3V 电源通过限流电阻连接，GPIO 端口负责灌电流，因此该 LED 为低电平点亮、高电平熄灭。

在实验代码中，`board_led_set(true)` 内部调用 `GPIO_ResetBits(GPIOA, GPIO_Pin_8)`，即把 PA8 拉低使 DS0 点亮；`board_led_set(false)` 则调用 `GPIO_SetBits(GPIOA, GPIO_Pin_8)`，把 PA8 拉高使 LED 熄灭。

3.5 按键检测

WK_UP 接在 PA0，上电后配置为下拉输入，所以未按下时读到低电平，按下后读到高电平。KEY1 接在 PA15、KEY0 接在 PC5，二者在实际工程中都配置为上拉输入，因此未按下时读到高电平，按下后读到低电平。这样三组按键在软件中分别判断为：

- PA0 == 1: KEY_UP 有效，发送 “SOS”
- PA15 == 0: KEY1 有效，发送 “YES”
- PC5 == 0: KEY0 有效，发送 “NO”

四、摩尔斯电码输出原理

4.1 摩尔斯电码基本规则

摩尔斯电码本质上是对“点”和“划”两种基本时长信号的编码。文档给出的国际标准规则是：1 个点 = 1 个时间单位，1 个划 = 3 个时间单位，字符内部符号间隔 = 1 个时间单位，字符之间间隔 = 3 个时间单位，单词之间间隔 = 7 个时间单位。摩尔斯灯语就是把这些时长关系映射到 LED 的亮灭时间上。

4.2 时间参数

参考文档允许 1 个时间单位在 50ms ~ 1000ms 之间选择；本工程将 1 个单位时间定义为 200ms，因此各符号时长如下：

符号	时长设置
点	200ms
划	600ms
符号间隔	200ms
字母间隔	600ms
单词间隔	1400ms

触发按键	发送内容	摩尔斯序列
KEY_UP	SOS	... --- ...
KEY1	YES	-.--
KEY0	NO	-. ---

4.4 输出过程与实现方法

实现方法如下：

先点亮 LED，保持对应时长；再熄灭 LED，保持一个符号间隔；若该字母结束，则补足字母间隔；若整条消息结束，则再补足单词间隔。

1. 消息输出

`emit_message()` 依次发送多个字母，消息全部结束后统一再延时 7 个单位，作为一次发送后的单词级空闲间隔，便于下次按键触发时观察。

2. 字母输出

`emit_letter()` 逐个读取字符串中的符号，如 "...", "---", "-.--", 循环调用

`emit_symbol()`。

由于 `emit_symbol()` 在每个符号末尾已经自动加入了 1 个单位的符号间隔, 所以字母结束后只需再补字母间隔 - 符号间隔, 即可保证最终字母间总间隔为 3 个单位。

3. 符号输出

`emit_symbol()` 负责输出单个 . 或 -。若符号是 ., 亮灯 200ms; 若符号是 -, 亮灯 600ms; 随后统一熄灯 200ms, 作为符号间隔。

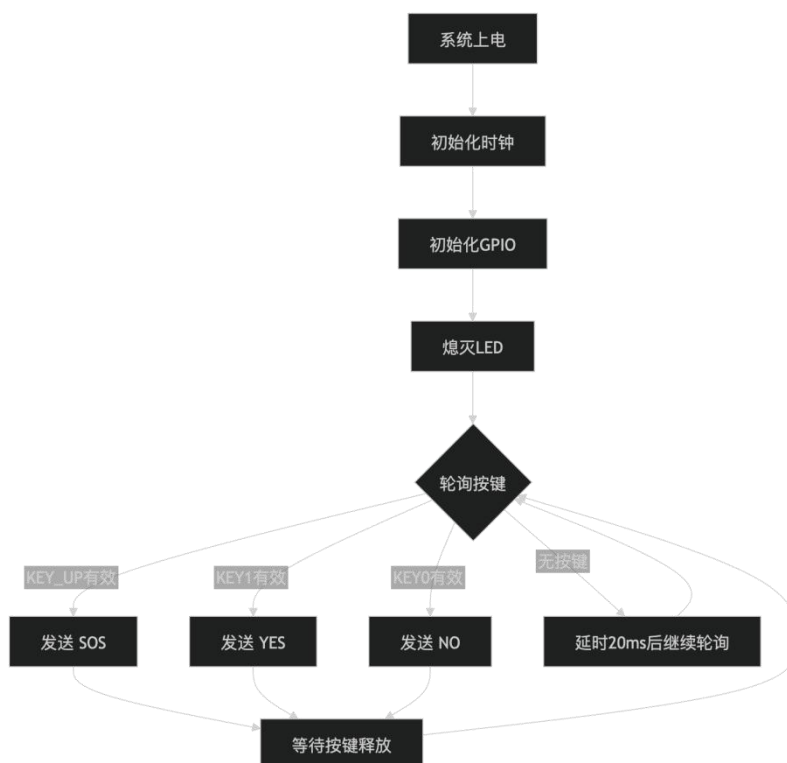
后续如果要继续扩展为发送任意字符串, 只需要增加字符到摩尔斯码的查表映射即可, 不必改底层灯光输出函数。

4.5 按键消抖与单次触发

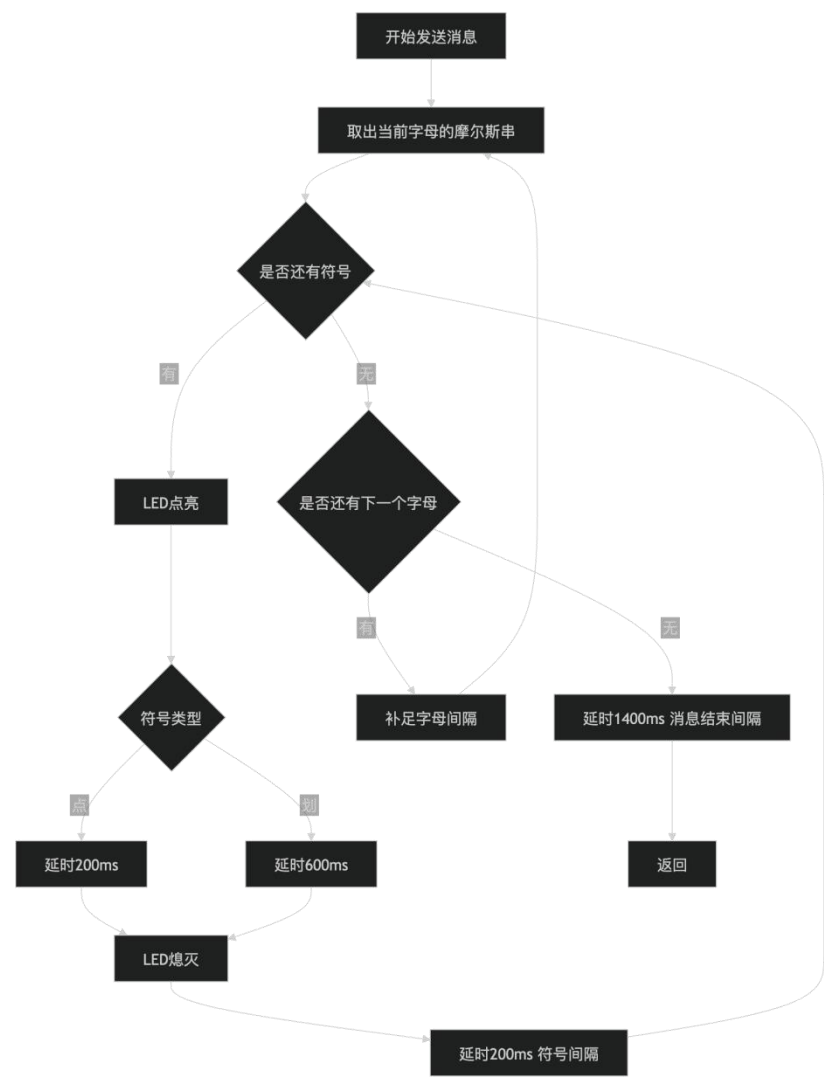
机械按键按下瞬间会出现抖动。如果直接检测端口电平, 一次按下可能被识别为多次触发。本工程处理方式如下:

1. `sample_key()` 先检测按键是否有效;
2. 若有效, 再延时 20ms 重新采样, 只有两次结果一致才认定为真正按下;
3. 一次消息发送完成后, 再通过 `wait_for_key_release()` 等待按键释放, 确保长按期间不会重复发送。

五、程序流程图



主程序流程图



单个消息发送流程图

六、主要程序分析

6.1 程序总体结构

文件	主要作用
main.c	仅调用 morse_app_run(), 作为程序入口
board.c	完成底层硬件抽象, 包括 GPIO 初始化、LED 控制、按键读取、毫秒延时
morse_app.c	完成相关逻辑, 包括摩尔斯码时序定义、消息发送、消抖和按键触发控制

6.2 底层初始化函数

`board_init()` 是底层初始化函数，主要完成以下工作：

1. 使能 GPIOA、GPIOC 和 AFIO 时钟；
2. 调用 `GPIO_PinRemapConfig(GPIO_Remap_SWJ_JTAGDisable, ENABLE)` 关闭 JTAG，释放 PA15；
3. 将 PA8 配置为推挽输出，用于控制 DS0；
4. 将 PA0 配置为下拉输入，用于读取 WK_UP；
5. 将 PA15 配置为上拉输入，用于读取 KEY1；
6. 将 PC5 配置为上拉输入，用于读取 KEY0。

6.3 LED 控制函数

`board_led_set(bool on)` 用于统一封装 LED 点亮与熄灭控制。

- 当 `on = true` 时，程序执行 `GPIO_ResetBits(GPIOA, GPIO_Pin_8)`，输出低电平，LED 点亮；
- 当 `on = false` 时，程序执行 `GPIO_SetBits(GPIOA, GPIO_Pin_8)`，输出高电平，LED 熄灭。

6.4 按键检测函数

- `board_key_up_pressed()`，判断 PA0 是否为高电平
- `board_key1_pressed()`，判断 PA15 是否为低电平
- `board_key0_pressed()`，判断 PC5 是否为低电平

七、实验步骤

- ① 分析软硬件原理、方法；
- ② 编写源程序；
- ③ 下载调试程序；
- ④ 修改源程序；
- ⑤ 撰写实验报告。

八、程序调试及结果分析

8.1 调试结果

操作	预期现象	结果分析
上电空闲	DS0 熄灭	程序初始化结束后调用 <code>board_led_set(false)</code> , 系统处于待触发状态
按下 KEY_UP	DS0 按 ... --- ... 闪烁一次	对应发送 “SOS”
按下 KEY1	DS0 按 -. -- 闪烁一次	对应发送 “YES”
按下 KEY0	DS0 按 -. --- 闪烁一次	对应发送 “NO”
持续按住任一按键	只发送一次, 不连续重复	发送完成后程序会等待按键释放

8.2 调试过程中重点问题及处理方法

(1) KEY1 无法正常触发

问题原因: PA15 默认被 JTAG 占用, 如果直接作为普通输入口使用, 按键读取可能异常。

处理方法: 在 `board_init()` 中增加
`GPIO_PinRemapConfig(GPIO_Remap_SWJ_JTAGDisable, ENABLE)`, 关闭 JTAG、保留 SWD 调试。

处理结果: KEY1 能稳定识别。该问题与指导书中 PA15 同时具备 JTDI/KEY1 复用功能是一致的。

(2) LED 亮灭逻辑与直觉相反

问题表现: 若直接把“输出高电平”理解为“点亮 LED”, 会发现现象与预期相反。

问题原因: DS0 为低电平点亮结构。

处理方法: 在 `board_led_set()` 中统一封装 LED 控制逻辑, 规定 `true` 表示“亮灯”, 内部映射为 `GPIO_ResetBits()`。

处理结果: 上层摩尔斯逻辑不再受硬件电平极性影响, 程序更清晰。

九、思考题

回答如下思考：GPIO 有哪些输出模式？结合本实验分析各种模式的特点及应用场景？

9.1 GPIO 的输入输出模式

根据 STM32F103 的 GPIO 定义，GPIO 常用的 8 种模式为：

类别	模式
输入模式	输入浮空、输入上拉、输入下拉、模拟输入
输出模式	开漏输出、推挽输出、复用推挽输出、复用开漏输出

9.2 结合本实验分析各种模式的特点与应用场景

1. 输入浮空 (GPIO_Mode_IN_FLOATING)

特点：输入端既不上拉也不下拉，端口电平完全由外部电路决定。

优点：适合外部已经有稳定驱动的数字信号输入。

缺点：若外部悬空，容易受干扰产生不确定电平。

应用场景：外部数字模块输出等。

本实验中未采用该模式，因为按键若使用浮空输入，未按下时电平不稳定，容易误判。

2. 输入上拉 (GPIO_Mode_IPU)

特点：端口内部接上拉电阻，默认读高电平，外部拉低时读低电平。

优点：适合“按下接地”的按键输入，硬件简单，稳定性好。

本实验应用：KEY1 (PA15) 和 KEY0 (PC5) 都配置为上拉输入，未按下时为高电平，按下后变为低电平，便于软件判断。

这是本实验最常用的输入模式之一。

3. 输入下拉 (GPIO_Mode_IPD)

特点：端口内部接下拉电阻，默认读低电平，外部拉高时读高电平。

优点：适合“按下接高电平”的输入场景。

本实验应用：WK_UP (PA0) 配置为下拉输入，未按下时读 0，按下后读 1。这样设计与 WK_UP 的硬件接法相匹配。

4. 模拟输入 (GPIO_Mode_AIN)

特点：关闭数字输入输出缓冲，供 ADC 等模拟外设使用。

优点：可减小数字噪声影响，适合模拟量采集。

应用场景：电压采样、传感器模拟信号输入。

本实验未涉及模拟信号，因此未采用。

5. 开漏输出 (GPIO_Mode_Out_OD)

特点：只能主动输出低电平，高电平通常依赖外部上拉。

优点：适合总线共享和电平转换。

应用场景：I²C 总线、多器件线与逻辑。

本实验未采用，因为 LED 驱动不需要总线共享，直接使用推挽输出更简单。

6. 推挽输出 (GPIO_Mode_Out_PP)

特点：既能主动拉高，也能主动拉低，驱动能力强，输出速度快。

优点：最适合普通数字输出控制。

本实验应用：PA8 配置为推挽输出，用于直接控制 DS0 亮灭。

这是本实验最核心的输出模式，因为摩尔斯灯语本质上就是通过 GPIO 反复输出高低电平形成时序。

7. 复用推挽输出 (GPIO_Mode_AF_PP)

特点：端口由片上外设接管，以推挽方式输出。

应用场景：USART 发送脚、PWM 输出脚、SPI 时钟/数据输出等。

本实验当前未用，但若后续想把摩尔斯内容通过 USART 串口同步发出，或者使用定时器 PWM 改进灯光输出，就会用到这种模式。

8. 复用开漏输出 (GPIO_Mode_AF_OD)

特点：端口由片上外设控制，但输出方式为开漏。

应用场景：I²C 外设引脚等需要开漏特性的复用功能场景。

本实验未使用。